Semantic Document Retrieval using Word Embeddings and Query Expansion

Manemoni Sumanjali (manemoni.sumanjali@iitgn.ac.in), Sidda Divya(24110337@iitgn.ac.in),
Atragada Venkata Naga Sai Ganesh (24110060@iitgn.ac.in), Divyam Sahu
CS328,Introduction to DataScience

***Abstract*—In this project we are building a search engine that can understand the meaning of a query, not just match exact keywords. The main idea is to use word embeddings (Word2Vec and BERT) to represent words and documents as vectors, then use Approximate Nearest Neighbor (ANN) search to first find related terms for the query and then retrieve relevant documents. We are trying three different designs and comparing them using Precision@k, Recall@k, F1@k, MRR@k, and NDCG@k to see which one gives better results.**

***Index Terms*—Word Embeddings, Semantic Search, Query Expansion, Approximate Nearest Neighbor Search, Document Retrieval, Word2Vec, BERT, Information Retrieval**

I. INTRODUCTION

Normal keyword based search has one big problem – it only matches exact words. For example, if someone searches for "automobile", documents that only have the word "car" will not show up, even though both mean the same thing. This is called the vocabulary mismatch problem.

Word embeddings solve this problem. The idea is that every word is converted into a vector of numbers, and words with similar meanings will have vectors that are close to each other. So "car" and "automobile" will be near each other in the vector space, and our search engine can use this to find relevant documents even when the exact words don't match.

In this project we are going to:

1) Build word embeddings for all words using Word2Vec and BERT.
2) For any given query, find the nearest neighbor words in the embedding space – these are the related terms.
3) Use the original query along with these related terms to retrieve documents.
4) Evaluate how well each design performs.

II. METHODOLOGY

2.1 Step 1: Build Word Embeddings

We are using two types of word embeddings:

Word2Vec [8]: Word2Vec basically trains a small neural network on the text data and learns which words appear near each other. Words appearing in similar contexts get similar vectors. For example, "king" and "queen" will end up close in vector space. We are training Word2Vec directly on the MS MARCO passages using Gensim [2] instead of using some pre-trained vectors, because that way the model learns word relationships that are specific to our dataset.

BERT [7]: BERT is a much more powerful pre-trained model. The main advantage of BERT over Word2Vec is that it gives different vectors for the same word depending on the context. For example, the word "bank" gets a different vector in "river bank" compared to "bank account". We are using a pre-trained Sentence-BERT model called all-MiniLM-L6-v2 from the sentence-transformers library [6] to convert full passages into single vectors.

2.2 Find Related Terms Using ANN Search

This is one of the key steps. Given a user query, we find related terms by doing the following:

1) Get the Word2Vec vector for each word in the query.
2) Run ANN search using FAISS [1] over all word vectors in the vocabulary.
3) The nearest neighbor words that come back are the related terms.

For example, if the query is "global warming", the related terms we might get are "climate change", "greenhouse gases", "rising temperatures", "carbon emissions" and so on.

After getting these related terms, we combine them with the original query words (removing any duplicates) and average all their Word2Vec vectors into one single vector. This is called the expanded query vector and it is used for document retrieval in the next step.

2.3 Retrieve Documents

Every document in our dataset is represented as a vector – either by averaging its Word2Vec word vectors or by encoding the full passage using Sentence-BERT. All these document vectors are stored in a FAISS index [1]. When a query comes in, FAISS does ANN search and returns the top-k most similar documents.

2.4 Step 4: Evaluate

We check how good the results are using five standard metrics [3] at $k = 5$ and $k = 10$:

- **Precision@k:**

$$\text{Precision@k} = \frac{\text{Number of relevant documents in top-}k}{k} \tag{1}$$

Precision@k measures how many of the top-k returned documents are actually relevant (useful).

- **Recall@k:**

$$\text{Recall@k} = \frac{\text{Number of relevant documents in top-}k}{\text{Total number of relevant documents}} \quad (2)$$

Recall@k measures how many of all the relevant documents were successfully retrieved.

- **F1@k:**

$$\text{F1@k} = \frac{2 \times \text{Precision@k} \times \text{Recall@k}}{\text{Precision@k} + \text{Recall@k}} \quad (3)$$

F1@k provides a single combined score that balances both precision and recall.

- **MRR@k** (Mean Reciprocal Rank) **:**

$$\text{MRR@k} = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (4)$$

MRR@k measures how quickly the system retrieves the first relevant document.

- **NDCG@k** (Normalized Discounted Cumulative Gain) **:**

$$\text{NDCG@k} = \frac{\text{DCG@k}}{\text{IDCG@k}} \quad (5)$$

$$\text{DCG@k} = \sum_{i=1}^{k} \frac{\text{rel}_i}{\log_2(i+1)} \quad (6)$$

NDCG@k gives higher scores when relevant documents appear higher in the ranking.

We are evaluating all five metrics for all three designs and comparing to see which one does better.

Speed Comparison: We are also going to measure how long each design takes to answer a query in milliseconds. This gives a practical idea of the speed vs accuracy tradeoff between the three designs.

Effect of Number of Related Terms: We will run experiments with N = 3, 5, and 10 related terms to see which gives the best results.

Qualitative Analysis: We will also manually look at the results for a few example queries to see what related terms were found and what the top results looked like for each design.

III. DATASET

We are using the MS MARCO v2.1 dataset [4, 5], which was made by Microsoft is using real Bing search queries. Each query has multiple passages, and human annotators have marked which passages are actually relevant. We chose this dataset because the relevance labels are already available, the queries are real search queries, which makes the evaluation more meaningful, and it is freely available on Hugging Face. We are using 1,000,000 passages, which is a good enough size to test the system properly while still being manageable on Google Colab.

Data Preprocessing: Before training Word2Vec, we are lowercasing everything, tokenising into individual words, and removing punctuation, and removing stopwords like "the", "is", "a", so that Word2Vec only focuses on words that actually carry meaning. For BERT, we don't need to do any of this since Sentence-BERT can handle raw text directly.

IV. SYSTEM OVERVIEW

4.1 Design 1: Word2Vec Based

- Word2Vec embeddings are used to expand the query with related terms via ANN search over the word vocabulary. The expanded query is then used to retrieve the top-k documents from the FAISS document index. See Appendix for the full diagram.

4.2 Design 2: BERT Based

- The query is encoded directly using Sentence-BERT into a single dense vector. FAISS runs ANN search over precomputed BERT passage embeddings to return the top-k documents. No query expansion is needed since BERT already captures the full sentence meaning.

4.3 Design 3: Hybrid (Word2Vec + BERT)

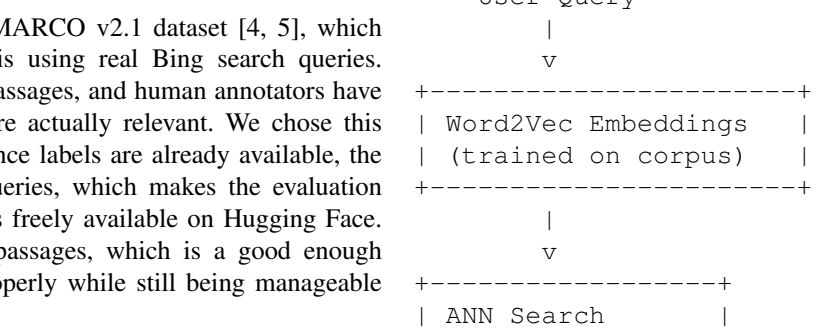
- Word2Vec with query expansion retrieves the top-100 candidates quickly via FAISS. Sentence-BERT then reranks only those 100 candidates to produce the final top-k results.

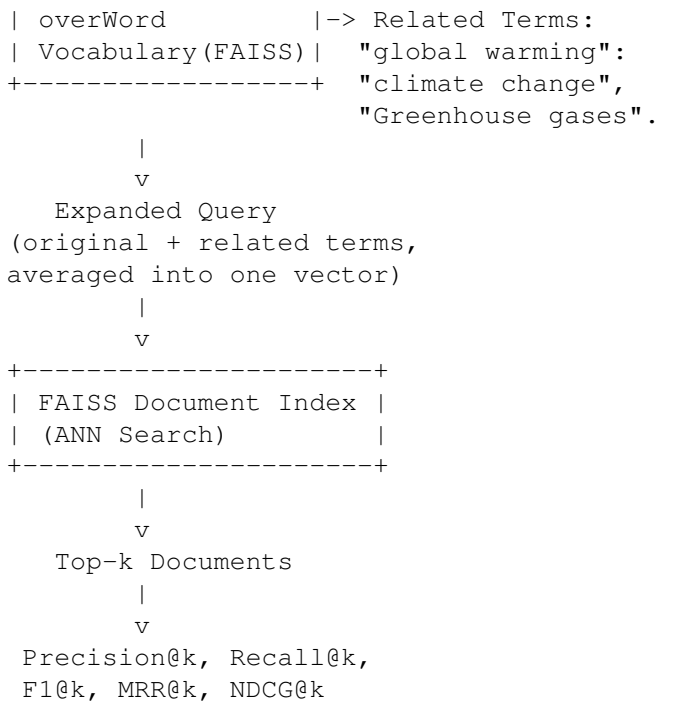
TABLE I
LIBRARIES USED IN THIS PROJECT

Library	Use
gensim	Train Word2Vec, find nearest neighbor words
sentence-transformers	Pre-trained BERT sentence embeddings
faiss-cpu	Approximate nearest neighbor search
datasets	Load MS MARCO
nltk	Tokenization and stopword removal
numpy	Vector operations
gradio	Web demo

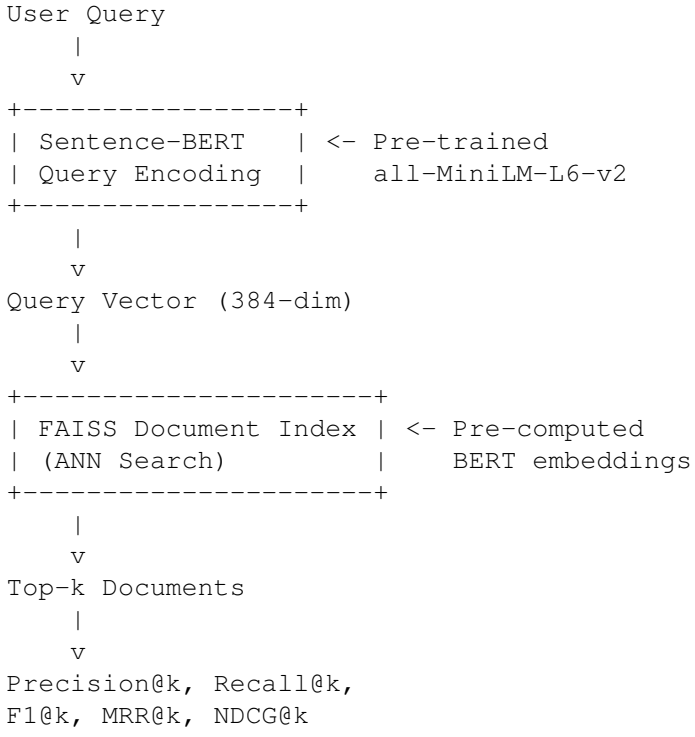
SYSTEM DIAGRAMS

A.1 Design 1: Word2Vec Based



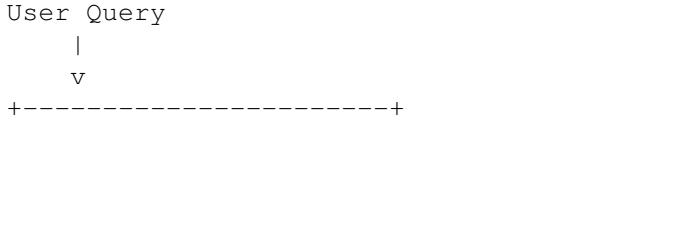


A.2 Design 2: BERT Based



Note: No query expansion for BERT since it already encodes the full sentence meaning.

A.3 Design 3: Hybrid (Word2Vec + BERT)



Note: BERT is only applied to 100 candidates, not all 1 million documents.

REFERENCES

[1] FAISS – Facebook AI Similarity Search. https://faiss.ai/
[2] Gensim Word2Vec Documentation. https://radimrehurek.com/gensim/models/word2vec.html
[3] Introduction to Information Retrieval – Precision and Recall. https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-in-informationretrieval-1.html
[4] MS MARCO Dataset. https://microsoft.github.io/msmarco/
[5] MS MARCO on Hugging Face. https://huggingface.co/datasets/microsoft/ms$_{marco}$
[6] Sentence Transformers Documentation. https://www.sbert.net/
[7] Understanding BERT – Google AI Blog. https://blog.google/products/search/search-language-understanding-bert/
[8] Word2Vec Explained – Towards Data Science. https://towardsdatascience.com/word2vec-explained-49c52b4ccb71